

Backend Requirements

Phase 2, Part 1 requirements definition — prepared for the Backend Engineer

1. Purpose and how to use this document

This document defines WHAT the Grade A+ backend must do in Phase 2 Part 1. It deliberately does not specify HOW. Your deliverable is a Backend System Architecture Document that answers the how, and it is due before any Part 1 implementation code is written.

Requirement identifiers are stable. Epics are lettered A–I and are shared across the Frontend, Backend and QA requirement documents, so BE-E-02, FE-E-03 and QA-E-01 all concern assignment weighting. Reference these identifiers in your architecture document so the QA engineer can build a traceability matrix against them.

Where a requirement says "must", it is binding. Where it says "should", propose an alternative if you have a better one. Where a requirement is marked as an assumption, it needs a decision before you design around it.

Why Part 1 exists. *Phase 2 was split because the original twelve-epic scope was not achievable in one window without either cutting quality or extending indefinitely. Part 1 is the set of work that has no external dependency — no marketplace approval, no third-party sandbox, no district contract. It should therefore finish on time. If you find an item here that does have an external dependency, raise it immediately, because it was placed in the wrong part.*

Field	Value
Document	Grade A+ Phase 2 (Part 1) Requirements
Version	0.1 — Draft for engineer review
Date	19 August 2026
Author / Owner	Founder
Primary audience	Backend Engineer
Delivery window	Approximately 4 months — September through end of December 2026
Expected deliverable	Backend System Architecture Document (data model, service boundaries, API contracts, job architecture, security model)
Status	Requirements baseline — no implementation code until the deliverable is reviewed and approved

2. System context

Grade A+ is an AI-assisted grading platform for K–12 and higher-education educators. Teachers create assignments and rubrics, collect digital or scanned paper submissions, grade them with AI assistance while retaining final control, and review analytics at the student, class and assignment level.

- Frontend: React.
- Backend: predominantly Python.
- Deployment: Railway (see the Deferred section — a migration remains under discussion and must not be assumed either way).
- LLM access: OpenRouter, with Grok as the primary model and GPT as the fallback. Model routing is dynamic; no component may hard-code a model.
- Identity: Sign in with Google is live for all user types.
- Metering: all AI usage is denominated in AI Credits.

Scope note. *Phase 2 has been split into two parts to control scope. This document covers Part 1 only. LMS integrations, District Standards, session archiving, student subgroups and equity indicators, and the platform-wide analytics agent are deliberately excluded and are recorded in the Deferred section. Do not design them in, but do not foreclose them either.*

3. Domain glossary

These definitions are binding. Where an existing implementation disagrees with a definition below, raise it rather than coding around it.

Term	Definition
Session	A school year or semester. Multiple Sessions may be active at the same time.
Course	A class. Every Course belongs to exactly one Session. Course enrolment is specific to a Session and does not carry forward.
Assignment	A gradeable artifact within a Course, with an associated rubric.
Lesson	A lesson plan owned by a teacher (or by a Department under a School License). Lessons are NOT scoped to a Session — they remain visible and usable across Sessions.
Tag	A teacher-applied label on an Assignment or Lesson representing the main topic or learning objective, used to group work for analytics. Three tag types exist in Part 1: Subject Name, Unit Name and Lesson.
Department	A named grouping of Licensed Teachers under a School License, created and administered by a School Admin. A Department persists across Sessions; its membership is faculty, not students.
Shared Assignment Library	A Department-scoped collection of Assignments that Department members may browse, add to, edit and copy from at zero AI credit cost.

Term	Definition
Submission	A student's work against an Assignment. Uploaded by the student (if invited) or by the teacher on the student's behalf.
AI Credit	The unit of AI consumption. Metered per teacher. Never pooled across teachers.
School License	A contract covering a set of Licensed Teachers, administered by a School Admin.

4. User roles

Role	Capabilities relevant to Phase 2 Part 1
Individual Teacher	Self-serve, tiered monthly subscription. Owns their own credit balance. Has a Lessons tab. No Department, and no Shared Assignment Library access.
Licensed Teacher	Managed teacher account under a School License. Own credit balance. Has a Lessons tab. May be placed by their admin into one or more Departments, and has no control over that placement.
School Admin	Manages the School License, teacher accounts and billing. Holds a separate credit balance and may purchase and disperse overage credits. Creates and administers Departments and Department Lessons. Receives aggregate AI insights and interventions.
Student	Added to a teacher account. Cannot self-register and cannot hold a subscription. May be invited to upload submissions. No visibility into Lessons, Departments or the Shared Assignment Library.
Super Admin	Internal. System analytics, School License creation, custom credit configuration, full audit log access.

5. Global constraints

5.1 Compliance

Grade A+ handles student education records. FERPA and COPPA compliance is a product-level requirement, not a feature. Every requirement in this document inherits the following constraints.

- Data minimisation: collect and transmit the least student data required for the feature to work.
- No student personally identifiable information in application logs, error traces, analytics events or third-party telemetry. Reference students by internal identifier only.
- Any transmission of student work or student identifiers to a third-party model provider must be deliberate, documented, and covered by the subprocessor disclosure the founder maintains.
- Deletion and export must remain possible for any student record on request.

- Students under 13 fall under COPPA; the platform relies on school or parental consent obtained through the teacher or School Admin, and must not create a path for a student to self-register.

5.2 Tenancy and authorisation

- Every data access path must be scoped by the requesting user's role and tenancy. A teacher sees only their own data; a School Admin sees only their own License; a Student sees only their own work.
- Department membership is an additional authorisation dimension in Part 1. Membership grants access to Department Lessons and the Shared Assignment Library, and grants nothing else — it must never expose another teacher's Courses, rosters, submissions or grades.
- Authorisation is enforced server-side. Hiding a control in the UI is never an authorisation mechanism.
- Cross-tenant leakage is treated as a P0 defect at any severity of exposure.

5.3 Credits

- Every AI invocation passes through a single, central metering service. No component may call a model directly.
- Operations designated zero-credit in this document must be provably zero-credit, enforced in the metering layer and covered by an automated test — not left to convention.
- Credit balances are per teacher and are never pooled.

5.4 Model independence

Grok is primary and GPT is the fallback, routed through OpenRouter. No prompt, parser, or feature may depend on the behaviour of one specific model. Every model interaction records the model and prompt version used.

6. Assumptions requiring confirmation

The following were not settled at the time of writing. Each is recorded as a working assumption so that design can proceed. Any of them may be overturned; flag early if an assumption materially changes your architecture.

#	Assumption	Impact if wrong
A1	Category weights within a Course must sum to 100%, enforced at save time.	Low.
A2	An Assignment or Lesson may carry more than one tag of each type.	Low.
A3	Tags are teacher-scoped for Individual Teachers and Department-scoped (shared) for Licensed Teachers within a Department.	Medium — changes the tag data model and the reuse behaviour.
A4	A Lesson belongs to one owner (a teacher or a Department) and is not co-owned. Sharing is	Medium — changes the Lessons data model.

#	Assumption	Impact if wrong
	achieved by placing it in a Department, not by multiple ownership.	
A5	A School Admin may prohibit specific Department members from editing Department Lessons or the Shared Assignment Library, at the level of a per-member permission flag rather than per-item ACLs.	Medium — per-item ACLs are materially more work.
A6	Audit log retention is 12 months for general events and 3 years for events touching student records.	Medium — storage design.
A7	Teacher edits to AI feedback replace the text a student sees, while the original AI text is retained internally and is not student-visible.	Low.
A8	Credit sufficiency estimation is advisory. A teacher may proceed past a "you may not have enough credits" warning.	Medium — a blocking estimate changes the job submission path.

Epic A — Action logging, error taxonomy and observability

Read first. *This epic is a prerequisite, not a feature. Every other epic in Part 1 produces events that this epic must capture, and the QA plan depends on being able to reconstruct what a user actually did. Sequence it first.*

BE-A-01 Emit a structured event for every meaningful user action. At minimum: authentication (success and failure), sign-out, grading request submitted, grading request completed or failed, assignment create / update / delete / copy, lesson create / update / delete, tag create / rename / delete, roster change, submission upload, credit transaction, department create / update / membership change, shared library add / edit / copy, admin action, data export, and permission change.

BE-A-02 Fix the event schema up front. Minimum fields: event ID, timestamp, actor ID, actor role, License ID, Department ID where applicable, action, target type, target ID, outcome, correlation ID, source IP, user agent, and a bounded metadata object.

BE-A-03 Propagate a correlation ID from the frontend request through the backend, any queued worker, and any model call, so a single user action is traceable end to end. A support conversation must be able to start from one identifier.

BE-A-04 Logs must contain no student personally identifiable information and no submission content in free-text fields. Identifiers only. See section 5.1.

BE-A-05 Classify every error into a fixed taxonomy — at minimum: user error, validation error, provider error, model error, and system error. Triage is impossible when they share a bucket, and the frontend cannot choose the right message without the classification.

BE-A-06 Define machine-readable reason codes for the common, recoverable user errors that today surface as generic failures. The QA document carries the authoritative list; at minimum it must cover missing or unmatched student name, student not on the roster, unreadable or corrupt file, unsupported file type, file too large, empty submission, missing rubric, and page or length limits exceeded.

- Each reason code carries a stable identifier, a plain-language message suitable for direct display, and a remediation hint.
- The set of reason codes is a shared contract with the frontend. Adding one is a coordinated change, not a backend-only change.

BE-A-07 Implement and enforce retention. See assumption A6; confirm before building.

BE-A-08 Provide a Super Admin query interface over the log, filterable by actor, role, action type, License, Department and time range. Provide a School Admin view scoped strictly to their own License.

BE-A-09 Emit alertable metrics for grading failure rate, model fallback rate, credit ledger anomalies, and the rate of each reason code from BE-A-06. A spike in one reason code is the earliest signal of a regression that users are experiencing but not reporting.

Epic B — Credits: warnings, estimation and out-of-credits behaviour

Public commitment. *The Grade A+ public FAQ states that if a user runs low on credits their work is never blocked or lost. The implemented behaviour must honour that sentence. Confirm the exact intended behaviour with the founder before designing the out-of-credits path.*

BE-B-01 All AI invocations pass through one central metering service. No component may reach a model provider directly.

BE-B-02 Maintain an append-only credit ledger. Every debit and credit records amount, reason, actor, related entity and resulting balance. Any balance must be reconstructable from the ledger alone.

BE-B-03 Provide pre-flight credit estimation for any operation that consumes credits, and specifically for batch operations: given a set of submissions, return an estimated cost before execution. Document the estimation method and state its accuracy tolerance.

BE-B-04 Return, alongside the estimate, an explicit sufficiency verdict against the requesting teacher's current balance — sufficient, marginal, or insufficient — together with the projected remaining balance. A number the user has to compare against their balance themselves is not the requirement.

BE-B-05 Implement threshold warnings as the balance depletes, and a defined out-of-credits behaviour consistent with the public FAQ commitment. Queueing or partial completion is acceptable; discarding submitted work is not.

BE-B-06 When a batch operation exhausts the balance partway through, the completed portion must be committed and durable, and the remainder must be recoverable without re-uploading. Report exactly where it stopped.

BE-B-07 Concurrency: two simultaneous grading jobs against a near-zero balance must not drive the balance negative. Specify the locking or reservation strategy in your architecture document.

BE-B-08 Allow a School Admin to purchase overage credits and disperse them to specific teachers on their License. Dispersal debits the admin and credits the teacher atomically; a partial application is a defect.

BE-B-09 Every credit-consuming operation must expose its actual post-execution cost through the API, not only its estimate. The gap between estimate and actual is a metric worth tracking; a persistently wrong estimate is worse than no estimate.

Epic C — Tags: Subject, Unit and Lesson

Design constraint. *Tags are the connective tissue for every analytics claim this product makes. If tagging is high-friction, teachers will not tag, and the insights in Epic H will be built on a sparse and biased sample. Design for reuse first and taxonomy purity second.*

BE-C-01 Implement three tag types applicable to both Assignments and Lessons: Subject Name, Unit Name, and Lesson. A tag represents the main topic or learning objective and exists to group work for analytics.

BE-C-02 Once created, a tag is reusable on future Assignments and Lessons without being re-typed. See assumption A3 for scoping.

BE-C-03 The Lesson tag references an actual Lesson entity, not a free-text string. This is the link that lets analytics relate graded outcomes back to what was taught.

BE-C-04 An Assignment or Lesson may carry multiple tags of each type. See assumption A2.

BE-C-05 Support rename, merge and delete of tags without orphaning the Assignments or Lessons that carry them. Deleting a tag in use must be explicit about the consequence and must state how many items are affected.

BE-C-06 Tags carry over on assignment copy, on lesson copy, and on addition to a Shared Assignment Library. A copy that loses its tags silently breaks the analytics chain.

BE-C-07 Expose all three tag types as first-class analytics dimensions — performance by Subject, by Unit and by Lesson, at student, course and (for admins) aggregate scope.

BE-C-08 Enforce a normalisation rule for tag names — at minimum case and whitespace — so that "Fractions" and "fractions " do not become two dimensions. Document the rule.

Epic D — Lessons

The defining property. *A Lesson is not scoped to a Session. This is the single most important thing to get right in this epic: Courses reset every term, Lessons do not. A teacher who rebuilds their lesson library every August will not use the feature twice.*

BE-D-01 Implement a Lesson entity owned by a teacher (or by a Department — see Epic F), visible and usable across every Session that owner has. Lessons must not be scoped to a Session or to a Course.

BE-D-02 Support two creation paths, mirroring Assignments: upload of an existing lesson document, and AI-assisted generation. The two paths must converge on the same stored entity, not two parallel shapes.

BE-D-03 AI-assisted lesson generation is credit-metered through the central service and respects the teacher's grade-level setting. Lesson upload is zero-credit.

BE-D-04 Lessons carry Subject Name and Unit Name tags per Epic C.

BE-D-05 Expose Lessons as retrievable context to the analytics and insight layer so that AI analytics can reference what was taught when explaining or predicting performance. Provide this as a retrieval interface returning the relevant subset, never as bulk lesson text injected into a prompt.

BE-D-06 Support copying a Lesson at zero credit cost, consistent with Epic E. A copy is a deep copy with no residual reference to the source.

BE-D-07 Support draft and published states and retain version history.

BE-D-08 Record and expose last-edited metadata on every Lesson: who edited it, and when. Under a Department this is a requirement rather than a nicety — see BE-F-06.

BE-D-09 Support export to PDF and DOCX. Schools use lesson plans in observation and evaluation processes and will require a portable artifact.

Epic E — Assignment functionality updates

BE-E-01 Support copying an Assignment and its rubric between Courses and between Sessions. The copy must be a deep copy with no residual reference to the source; editing either must never mutate the other.

- Copy operations consume zero AI credits. This must be enforced by an assertion in the metering layer and covered by an automated test, not left to convention.
- A copy carries over the rubric, category assignment, weight, tags and grade level. A copy must never carry student data, submissions, grades or feedback.
- Support bulk copy of multiple assignments in one operation — this is the real workflow at session rollover — with partial-success semantics.

BE-E-02 Support named assignment categories per Course, each carrying a percentage weight (for example Exam 50%, Homework 30%). Support teacher-level and School-level category templates that seed a new Course.

BE-E-03 Validate that category weights sum to 100% at save time. See assumption A1. Define and document the treatment of assignments that have no category.

BE-E-04 Support per-assignment weight variation inside a category, so a final exam can outweigh a midterm within the same Exam category. Model this as a relative weight or point value within the category rather than as a second percentage system.

BE-E-05 Implement weighted average calculation as a single shared service. The gradebook, analytics and exports must all call it. Duplicated grade maths in more than one place is a defect.

BE-E-06 Recalculation must be deterministic and idempotent. A weight change recomputes affected averages and records the change in the audit log with before and after values.

BE-E-07 Define explicit, documented treatment for excused, missing, and not-yet-graded submissions in the weighted average. Silent exclusion is not acceptable; the chosen behaviour must be discoverable through the API.

BE-E-08 Allow a teacher to edit AI-generated feedback text before it reaches a student. Persist the original AI text and the teacher-edited text as a pair; never overwrite the original. See assumption A7.

- Every edit is a labelled training signal for improving grading accuracy. Store it in a shape that a future accuracy effort can query — original, edited, assignment, rubric criterion, strictness level, prompt version.
- Editing feedback is zero-credit.

BE-E-09 AI-assisted assignment creation must accept an existing Lesson as a reference input, so a generated assignment is grounded in what was actually taught. Pass the Lesson through the same retrieval interface as BE-D-05; do not concatenate the whole lesson into the prompt.

BE-E-10 Assignments carry Subject Name, Unit Name and Lesson tags per Epic C.

Epic F — Departments: School Admin

Read first. *A Department is a faculty grouping that outlives a Session. Courses reset every term because the students change; Departments do not, because the teachers do not. Modelling a Department as session-scoped would be the most expensive mistake available in this epic.*

BE-F-01 Implement a Department entity scoped to a School License, with a name and a teacher roster. A Department persists across Sessions and is not scoped to one.

BE-F-02 Only a School Admin may create, rename, delete a Department or change its membership. Members are selected from the set of Licensed Teachers on that License.

BE-F-03 A Licensed Teacher may belong to zero, one or many Departments. Membership is assigned by the admin; the teacher has no control over it and no self-service path to join or leave.

BE-F-04 Implement Department Lessons: a Lesson collection scoped to the Department. Every member and the admin may access, add and edit lessons in it, subject to the per-member restriction in BE-F-07.

BE-F-05 A School Admin may create Lessons in the Department Lessons collection.

BE-F-06 Every Department Lesson and every Shared Assignment Library entry must record and expose who last edited it and when. In a shared editing surface this is what makes the surface usable at all — without it, no teacher can tell whether the version in front of them is the one they wrote.

BE-F-07 Support a School Admin restricting a specific member's ability to add or edit Department Lessons or Shared Assignment Library entries. See assumption A5. Enforce server-side; the default is that members may edit.

BE-F-08 Implement the Shared Assignment Library: an Assignment collection scoped to the Department. Members may access, add, edit and copy from it.

- A School Admin may not create assignments in the Shared Assignment Library in Part 1. This is deliberate and is expected to change later — do not model it as a hard-coded role prohibition that would be expensive to relax.

BE-F-09 All copying into and out of the Shared Assignment Library is zero-credit, enforced in the metering layer and covered by an automated test.

BE-F-10 Department membership grants access to Department Lessons and the Shared Assignment Library and nothing else. It must never expose another teacher's Courses, rosters, submissions, grades or credit balance. This is the primary tenancy risk introduced in Part 1.

BE-F-11 All Department create, rename, delete and membership changes are audit-logged with actor, role, timestamp and target, per Epic A.

BE-F-12 Define and document what happens to Department Lessons and Shared Assignment Library entries when a teacher is removed from a Department or leaves the License. Content must not silently disappear for the remaining members.

Epic G — Departments: Licensed Teacher

BE-G-01 Expose, for an authenticated Licensed Teacher, the set of Departments they have been placed in, with no create, join or leave capability.

BE-G-02 Expose the member roster of each Department the teacher belongs to — names and role only. No credit balances, no course lists, no student data.

BE-G-03 Provide read, add and edit access to Department Lessons for members, subject to the per-member restriction in BE-F-07.

BE-G-04 Provide read, add, edit and copy access to the Shared Assignment Library for members, subject to the same restriction.

BE-G-05 Copying an Assignment from the Shared Assignment Library into a teacher's Course, and copying one of a teacher's own Assignments into the Library, are both zero-credit. Assert in the metering layer.

BE-G-06 A copy taken from the Library is independent of the Library entry. A later edit to the Library entry must not alter copies teachers have already taken, and an edit to a teacher's copy must not alter the Library entry.

BE-G-07 A teacher who is in no Department must receive an empty, well-formed response — not an error, and not a permissions failure.

BE-G-08 Individual Teachers have no Department and no Shared Assignment Library. Enforce server-side; absence of the UI is not enforcement.

Epic H — School Admin AI insights and interventions

Highest-risk epic. *This epic touches student records, algorithmic recommendation and aggregate reporting at the same time. Treat every requirement here as compliance-bearing. Where you are unsure, choose the more conservative option and raise it.*

BE-H-01 Provide a School Admin with aggregate analytics across the teachers on their License, drawing on lessons, assignments, tags and grades. Scope is derived from the authenticated admin's License, never from prompt instructions.

BE-H-02 Interventions are suggestions to a human. The system must never take an automated action against a student record, a teacher account, or a grade on the basis of an AI insight.

BE-H-03 All numeric aggregation is performed in Python or SQL. The model narrates results; it must never compute them. A statistic that appears in an AI answer must be traceable to a deterministic query.

BE-H-04 The insight layer must not receive bulk raw data in its prompt. Provide a tool interface it calls on demand, returning compact structured aggregates. Suggested minimum tools: performance by tag, performance by lesson, course summary, assignment outliers, teacher-level rollup.

BE-H-05 Enforce a minimum group size before any aggregate statistic is returned, to prevent re-identification of an individual student. Recommend a floor of five.

BE-H-06 Persist, for every generated insight or intervention: the source data snapshot identifier, model, prompt version and generation timestamp. An insight an admin may repeat in a staff meeting must be reconstructable.

BE-H-07 Insight and intervention generation is credit-metered against the admin's own balance, and the cost is exposed to the caller before and after execution, per Epic B.

BE-H-08 Record admin disposition of each intervention — acted on, dismissed, ignored — as a structured field. This is the data that will later tell you whether interventions are worth anything.

BE-H-09 The insight layer must function equivalently on the fallback model. Record model and prompt version on every response.

Epic I — Grading engine: accuracy and adjustable strictness

BE-I-01 Implement adjustable grading strictness as a bounded, discrete scale (for example Lenient, Standard, Strict). Free-text strictness instructions are not acceptable — they are untestable.

BE-I-02 Each strictness level maps to a versioned, stored grading configuration. Changing a level's behaviour creates a new version; it never mutates an existing one.

BE-I-03 Support strictness at teacher default, Course and per-Assignment scope, with a documented precedence order.

BE-I-04 Every graded submission records the prompt version, grading configuration version, strictness level and model used. Without this, accuracy regression testing is impossible and the QA accuracy suite cannot function. Sequence this early.

BE-I-05 Support re-grading a submission at a different strictness without destroying the prior result. Retain both and mark which is authoritative.

BE-I-06 Treat prompt and grading configuration changes as code changes: version-controlled, reviewed, and released through the same gate as application code.

BE-I-07 Submission content must never be treated as instruction. Text inside a student submission that resembles a directive to the grader must not influence the grade. Structure the prompt so that submission content is unambiguously data.

BE-I-08 Expose grading confidence or a comparable signal where the model output supports it, so that low-confidence results can be routed to teacher review rather than presented with the same authority as high-confidence ones.

Deferred — Phase 2 Part 2 and beyond

The following are out of scope for Part 1. They are recorded so that you can avoid design decisions that would make them harder later. Do not implement them, and do not assume an outcome.

Ref	Item	What to avoid foreclosing
D-01	LMS integrations (Google Classroom, Canvas, Schoology)	Keep external identity mapping possible: do not assume Grade A+ identifiers are the only identifiers a Course, Assignment or Student will ever have.
D-02	District Standards ingestion and alignment	Keep the retrieval interface built for Lessons (BE-D-05) general enough to serve a second corpus without redesign.
D-03	Session archiving and read-only enforcement	Avoid assuming every Session is writable. Do not scatter write paths that would each need their own guard later.
D-04	Student subgroups and equity indicators	Do not ingest, infer or store protected-class attributes in Part 1 under any circumstance.
D-05	Platform-wide analytics agent for teachers	Build the Epic H tool interface so a teacher-scoped caller can reuse it rather than needing a parallel implementation.
D-06	Migration away from GitHub or Railway	Avoid platform-specific primitives for state, cron or networking. Containerise cleanly and keep configuration external.
D-07	Flux as an image-generation sub-agent	Keep the model routing layer capable of non-text modalities; do not hard-code a text-only contract.
D-08	Additional assignment types	Model assignment type as data, not as an enum branching through the grading pipeline.
D-09	School Admin authoring in the Shared Assignment Library	See BE-F-08. Do not encode the current prohibition as an immovable role rule.

Your deliverable

Produce a Backend System Architecture Document before writing Part 1 implementation code. It should contain, at minimum:

- Data model changes: entity relationship diagram covering Lessons, Departments, Department Lessons, the Shared Assignment Library, tags, categories and weights, the credit ledger, and audit events.
- Service boundaries: which concerns are separated, and specifically where the credit metering service, the grade calculation service and the retrieval interface sit.
- API contracts for every new or changed endpoint, including error codes and the machine-readable reason codes required by BE-A-06.

- Job and queue architecture for batch grading and AI generation, including retry, backoff and partial-failure semantics.
- Authorisation model: the full role-by-resource matrix including the Department dimension, and where enforcement lives. BE-F-10 is the highest-risk line in this document; treat it explicitly.
- Migration plan and backwards-compatibility strategy for data created before Part 1, including how existing Unit tags map onto the new three-type tag model.
- A dependency-ordered sequencing proposal. Action logging (Epic A), credit metering (Epic B) and prompt versioning (BE-I-04) are prerequisites for several other epics and for the QA accuracy suite; say so explicitly if you agree.
- Your own estimate per epic against the four-month window, and a list of the requirements you consider unachievable in it.

Push back where warranted. *This document describes nine epics in a four-month window. Part 1 exists precisely so that the scope is honest. If nine still exceeds what is achievable, say so in your architecture document with your recommended cut, rather than starting all of them.*